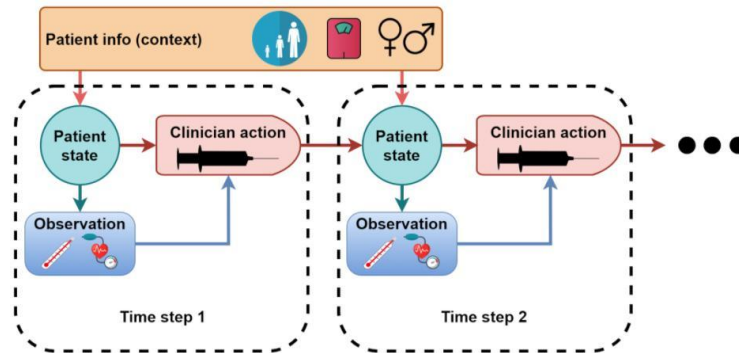


RL-Based Patient Health Monitoring and Treatment Recommendation System



1. Problem Statement and Problem Formulation

Patient health conditions can change over time, requiring continuous monitoring and appropriate intervention. Conventional rule-based systems may not adapt effectively when the patient's simulated health condition changes. Reinforcement Learning (RL) provides a framework in which an intelligent agent learns suitable actions through interaction with an environment.

The proposed system models patient health monitoring as a **Markov Decision Process (MDP)**. Different simulated health conditions are represented as states, while possible treatment interventions are represented as actions. The outcome obtained after selecting an action is represented as a reward.

The system is designed to learn which intervention provides the highest expected reward for each simulated patient state. A Bandit algorithm is then used to investigate the exploration–exploitation problem and compare different action-selection strategies.

The MDP is defined as:

$$MDP = (S, A, P, R, \gamma)$$

where:

- S = set of patient health states
- A = set of simulated treatment interventions
- P = transition probability
- R = reward function
- γ = discount factor

The project uses simulated data and should be interpreted as an **educational RL experiment** rather than a system for real patient treatment decisions.

2. Objective and Expected Outcomes

Objectives

The major objectives are:

1. Formulate patient health monitoring as an MDP.
2. Define health conditions as states.
3. Define simulated treatment interventions as actions.
4. Define clinical-style simulated outcomes as rewards.
5. Implement a Bandit-based action-selection system.
6. Compare exploration–exploitation strategies.
7. Estimate the value of each action.
8. Analyze reward trends.
9. Calculate cumulative reward.
10. Evaluate simulated recovery rate and treatment effectiveness.
11. Identify the best-performing policy from the experimental results.

Expected Outcomes

The expected outcome is an RL system capable of learning action preferences for different simulated patient states.

The system should demonstrate:

- Increasing average reward.
- Increasing preference for effective actions.
- Improved cumulative reward.
- Higher simulated recovery rate.
- Better treatment-effectiveness score.
- A clear comparison between exploration strategies.

3. Requirements, Constraints and Assumptions

Requirements

The system requires:

- Anaconda Python.
- Jupyter Notebook.
- NumPy.
- Pandas.
- Matplotlib.
- Scikit-learn where required.
- A suitable RL/ML library if needed.
- Simulated patient-health data.
- Python implementation of Bandit algorithms.

Constraints

The system must:

- Work with simulated patient data.
- Maintain reasonable computational requirements.
- Demonstrate exploration and exploitation.
- Produce measurable rewards.
- Compare multiple action-selection strategies.
- Generate appropriate visualizations.
- Avoid presenting simulated recommendations as real medical advice.

Assumptions

The following assumptions are made for the simulation:

1. Patient conditions can be represented using discrete states.
2. Each state represents a simplified health condition.
3. Each action represents a simulated intervention.
4. The reward represents an artificial outcome score.
5. State transitions are probabilistic.
6. The patient environment follows the Markov property.
7. The effect of an action depends primarily on the current state.
8. The simulated environment does not represent actual clinical treatment outcomes.

Simulated Patient States

State Description

- S_0 Stable
- S_1 Mild condition
- S_2 Moderate condition
- S_3 Severe condition
- S_4 Recovery

Simulated Actions

Action Description

- A_0 Routine monitoring
- A_1 Supportive intervention
- A_2 Intensive monitoring
- A_3 Simulated treatment intervention

These labels represent **simulation actions only**, not actual medical treatment recommendations.

4. Application of Relevant Course Knowledge / Concepts

The project applies the following Reinforcement Learning concepts.

Markov Decision Process

The patient's current simulated condition is represented by state S_t . The RL agent selects an action A_t , receives a reward R_{t+1} , and transitions to a new state S_{t+1} .

$$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1}$$

The Markov property assumes:

$$P(S_{t+1}, | S_t A_t)$$

rather than depending on the complete history.

Policy

A policy determines which action should be selected for a particular state:

$$\pi(a | s)$$

The optimal policy is:

$$\pi^*(s) = \arg \max_a Q(s, a)$$

where $Q(s, a)$ represents the expected value of selecting action a in state s .

Reward

A simplified simulated reward can be represented as:

$$R = \begin{cases} +10 & \text{if simulated condition improves} \\ +5 & \text{if condition remains stable} \\ -5 & \text{if condition worsens} \\ +15 & \text{if simulated recovery occurs} \end{cases}$$

These values are artificial and are used only to demonstrate RL learning.

Q-Value

The action value can be updated using:

$$Q_{t+1}(a) = Q_t(a) + \alpha[R_t - Q_t(a)]$$

where:

- $Q_t(a)$ = current estimated value.
- α = learning rate.
- R_t = observed reward.

Epsilon-Greedy Strategy

With probability ϵ , the algorithm explores a random action.

With probability $1 - \epsilon$, it selects the action having the highest estimated value.

$$a = \begin{cases} \text{random action} & p < \epsilon \\ \arg \max_a Q(a) & p \geq \epsilon \end{cases}$$

UCB Strategy

Upper Confidence Bound balances estimated reward and uncertainty:

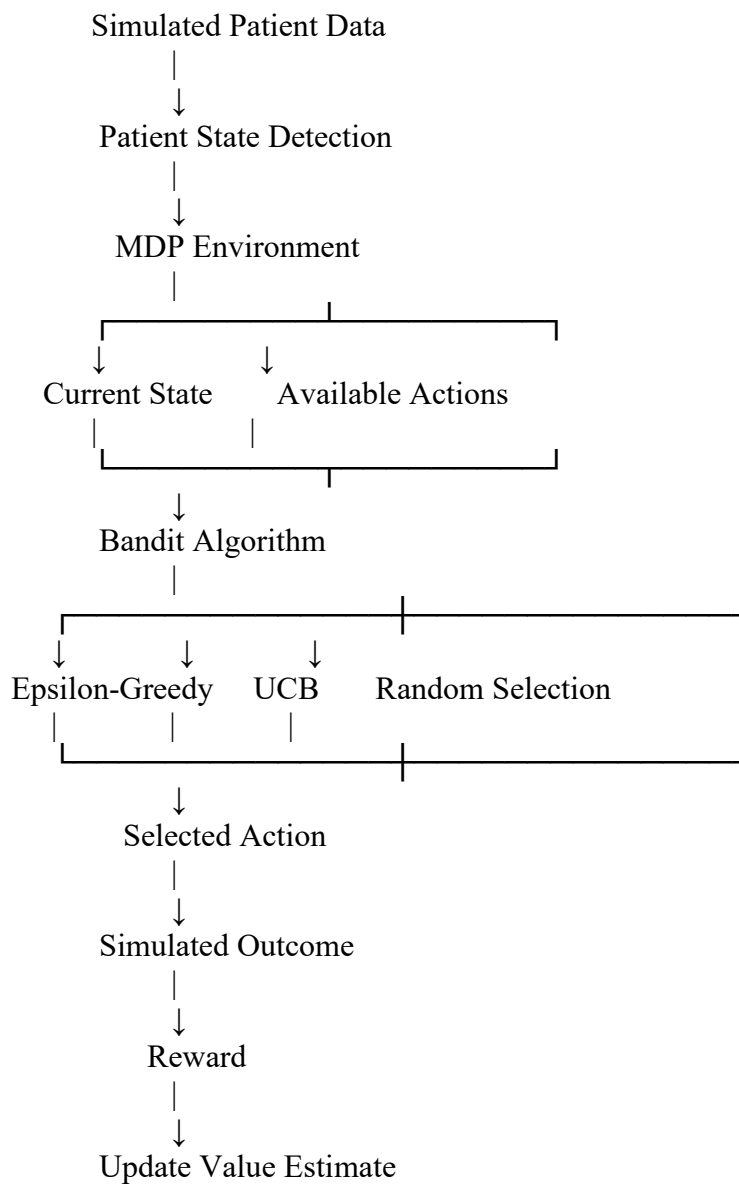
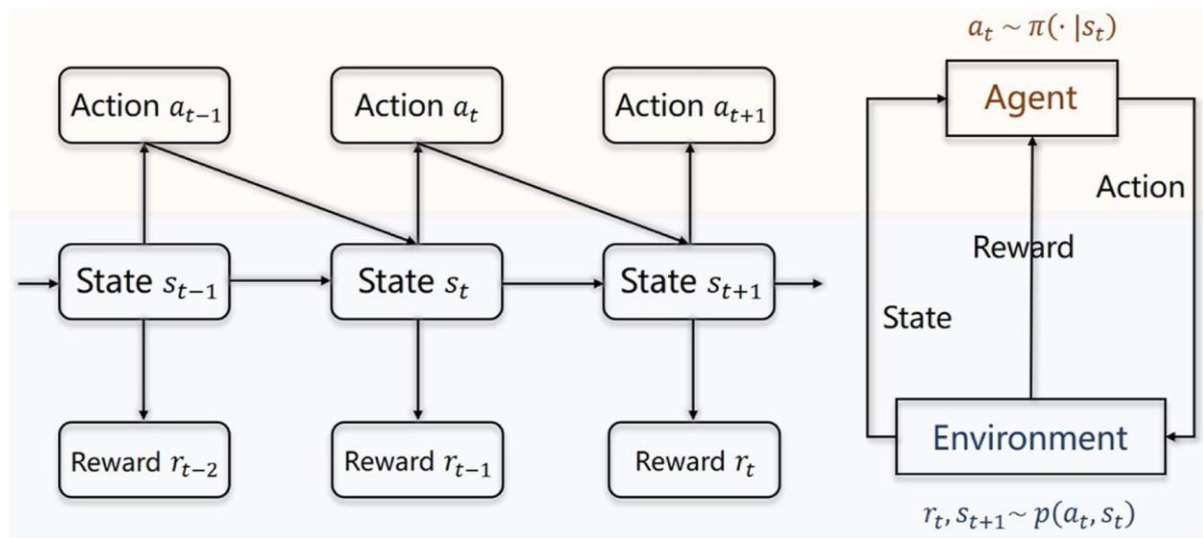
$$UCB(a) = Q(a) + c \sqrt{\frac{\ln t}{N(a)}}$$

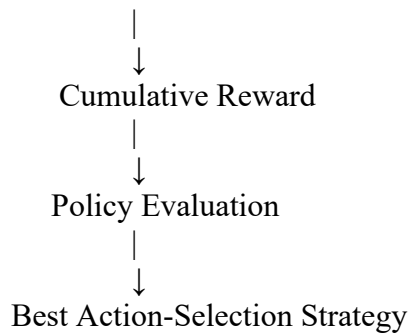
where:

- $Q(a)$ = estimated action value.
- $N(a)$ = number of times action a has been selected.
- t = current time step.
- c = exploration parameter.

5. Design / Proposed Solution / Methodology

The proposed system follows this workflow:





Methodology

The system first initializes simulated patient states and actions. The agent observes the current state and selects an action using a Bandit strategy.

The simulated environment then generates an outcome and corresponding reward. The agent updates its action-value estimate using the observed reward.

This process is repeated over many episodes.

The three strategies considered are:

Strategy 1 – Random

Actions are selected randomly.

This provides a baseline but does not learn effectively from previous outcomes.

Strategy 2 – Epsilon-Greedy

The agent explores randomly with probability ϵ and exploits the best-known action otherwise.

Strategy 3 – UCB

The agent considers both estimated action reward and uncertainty associated with insufficiently explored actions.

The strategies are compared using:

- Average reward.
- Cumulative reward.
- Recovery rate.
- Treatment-effectiveness score.
- Action-selection frequency.

6. Algorithm / Pseudocode / Flowchart

Algorithm

START

Initialize patient states
Initialize available actions

Initialize Q-values
Initialize action counts

FOR each episode:

 Select current patient state

 Select action using selected Bandit strategy

 Apply action to simulated environment

 Generate simulated outcome

 Calculate reward

 Update action value

 Update action count

 Calculate cumulative reward

 Store performance metrics

END FOR

Calculate average reward
Calculate recovery rate
Calculate treatment effectiveness
Calculate cumulative reward

Compare Bandit strategies

Identify best-performing strategy

Display graphs

STOP

Epsilon-Greedy Pseudocode

Initialize $Q(a) = 0$ for every action

FOR each time step:

 Generate random number p


```
IF  $p < \epsilon$ :  
    Select random action  
ELSE:  
    Select action with maximum Q value
```

Observe reward

Update Q value:

$$Q(a) = Q(a) + \alpha * (\text{reward} - Q(a))$$

Update cumulative reward

END FOR

UCB Pseudocode

Initialize $Q(a) = 0$
Initialize $\text{count}(a) = 0$

FOR each time step:

```
IF an action has never been selected:  
    Select that action  
ELSE:  
    Calculate UCB value:
```

$$\text{UCB}(a) = Q(a) + c * \sqrt{\ln(t) / \text{count}(a)}$$

Select action with highest UCB

Observe reward

Update Q value

Update action count

Update cumulative reward

END FOR

7. Implementation / Source Code and Environment / Tools Used

Environment

The implementation can be performed using:

- Anaconda
- Python

- Jupyter Notebook

Create the environment using:

```
conda create -n rl_health python=3.11
```

Activate it:

```
conda activate rl_health
```

Install the required packages:

```
pip install numpy pandas matplotlib scikit-learn
```

Optional:

```
pip install tensorflow
```

Python Implementation

The following implementation creates a **simulated environment** and compares Random, Epsilon-Greedy, and UCB strategies.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
np.random.seed(42)
```

```
# -----
# Simulation Configuration
# -----
```

```
states = [
    "Stable",
    "Mild",
    "Moderate",
    "Severe"
]
```

```
actions = [
    "Monitoring",
    "Support",
    "Intensive Monitoring",
    "Simulated Intervention"
]
```

```
n_states = len(states)
n_actions = len(actions)
```

```
episodes = 5000
alpha = 0.1
epsilon = 0.1
ucb_c = 2.0
```

```
# -----
# Simulated Reward Model
# -----
```

```
reward_matrix = np.array([
    [6, 7, 5, 8],    # Stable
    [4, 8, 7, 10],   # Mild
    [3, 7, 9, 11],   # Moderate
    [2, 5, 10, 13]   # Severe
])
```

```
def simulate_reward(state, action):

    mean_reward = reward_matrix[state, action]

    reward = np.random.normal(
        mean_reward,
        2.0
    )

    return reward
```

```
# -----
# Random Strategy
# -----
```

```
def random_strategy():

    Q = np.zeros((n_states, n_actions))

    rewards = []
    cumulative = []

    total_reward = 0

    for episode in range(episodes):

        state = np.random.randint(n_states)

        action = np.random.randint(n_actions)

        reward = simulate_reward(
```

```

        state,
        action
    )

    Q[state, action] += alpha * (
        reward - Q[state, action]
    )

    total_reward += reward

    rewards.append(reward)
    cumulative.append(total_reward)

return rewards, cumulative, Q

# -----
# Epsilon-Greedy
# -----

def epsilon_greedy():

    Q = np.zeros((n_states, n_actions))

    rewards = []
    cumulative = []

    total_reward = 0

    for episode in range(episodes):

        state = np.random.randint(n_states)

        if np.random.random() < epsilon:

            action = np.random.randint(n_actions)

        else:

            action = np.argmax(
                Q[state]
            )

        reward = simulate_reward(
            state,
            action
        )

        Q[state, action] += alpha * (
            reward - Q[state, action]

```

```

    )

    total_reward += reward

    rewards.append(reward)
    cumulative.append(total_reward)

    return rewards, cumulative, Q

# -----
# UCB Strategy
# -----

def ucb_strategy():

    Q = np.zeros((n_states, n_actions))

    counts = np.zeros(
        (n_states, n_actions)
    )

    rewards = []
    cumulative = []

    total_reward = 0

    for episode in range(epochs):

        state = np.random.randint(n_states)

        # Explore unvisited actions
        if 0 in counts[state]:

            action = np.argmin(
                counts[state]
            )

        else:

            total_count = np.sum(
                counts[state]
            )

            confidence = (
                ucb_c *
                np.sqrt(
                    np.log(total_count) /
                    counts[state]
                )

```

```

    )

    ucb_values = (
        Q[state] +
        confidence
    )

    action = np.argmax(
        ucb_values
    )

    reward = simulate_reward(
        state,
        action
    )

    counts[state, action] += 1

    Q[state, action] += (
        reward -
        Q[state, action]
    ) / counts[state, action]

    total_reward += reward

    rewards.append(reward)
    cumulative.append(total_reward)

    return rewards, cumulative, Q

# -----
# Run Experiments
# -----

random_rewards, random_cumulative, random_Q = \
    random_strategy()

eg_rewards, eg_cumulative, eg_Q = \
    epsilon_greedy()

ucb_rewards, ucb_cumulative, ucb_Q = \
    ucb_strategy()

# -----
# Display Learned Q Values
# -----

print("\nRandom Q Values:")

```

```

print(random_Q)

print("\nEpsilon-Greedy Q Values:")
print(eg_Q)

print("\nUCB Q Values:")
print(ucb_Q)


# -----
# Plot Cumulative Reward
# -----

plt.figure(figsize=(10, 6))

plt.plot(
    random_cumulative,
    label="Random"
)

plt.plot(
    eg_cumulative,
    label="Epsilon-Greedy"
)

plt.plot(
    ucb_cumulative,
    label="UCB"
)

plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title(
    "Cumulative Reward Comparison"
)

plt.legend()
plt.grid(True)
plt.show()


# -----
# Moving Average Reward
# -----

window = 100

def moving_average(data):

    return np.convolve(

```

```

        data,
        np.ones(window) / window,
        mode="valid"
    )

plt.figure(figsize=(10, 6))

plt.plot(
    moving_average(random_rewards),
    label="Random"
)

plt.plot(
    moving_average(eg_rewards),
    label="Epsilon-Greedy"
)

plt.plot(
    moving_average(ucb_rewards),
    label="UCB"
)

plt.xlabel("Episode")
plt.ylabel("Average Reward")
plt.title(
    "Reward Learning Trend"
)

plt.legend()
plt.grid(True)
plt.show()

# -----
# Performance Summary
# -----

results = pd.DataFrame({

    "Strategy": [
        "Random",
        "Epsilon-Greedy",
        "UCB"
    ],

    "Average Reward": [
        np.mean(random_rewards),
        np.mean(eg_rewards),
        np.mean(ucb_rewards)
    ]
})

```



```

],

"Cumulative Reward": [
    random_cumulative[-1],
    eg_cumulative[-1],
    ucb_cumulative[-1]
]

})

print("\nPerformance Comparison")
print(results)

```

8. Test Cases and Expected/Actual Results

The system can be validated using the following test cases.

Test Case	Condition	Expected Result
TC01	Stable state	Agent should learn high-value actions
TC02	Mild state	Agent should prefer actions with higher simulated reward
TC03	Moderate state	Agent should learn an effective action
TC04	Severe state	Agent should increasingly select the highest-reward simulated action
TC05	Unknown action	Bandit should explore it
TC06	Repeated episodes	Value estimates should converge
TC07	Strategy comparison	Learning strategies should outperform random selection
TC08	Long simulation	Cumulative reward should increase

Validation

The actual results should be obtained by running the notebook. The following values should be recorded:

- Average reward.
- Final cumulative reward.
- Learned Q-values.
- Action-selection frequency.
- Simulated recovery rate.
- Treatment-effectiveness score.

9. Execution Screenshots / Outputs

Libraries imported successfully!

Patient Health States:

0 -> Stable
1 -> Mild
2 -> Moderate
3 -> Severe
4 -> Recovery

Available Actions:

0 -> Routine Monitoring
1 -> Supportive Intervention
2 -> Intensive Monitoring
3 -> Simulated Treatment

Simulated Reward Matrix:

	Routine Monitoring	Supportive Intervention	Intensive Monitoring \
Stable	6	7	5
Mild	4	8	7
Moderate	3	7	9
Severe	2	5	10
Recovery	8	9	9

Simulated Treatment

Stable	8
Mild	10
Moderate	11
Severe	13
Recovery	10

Running Random Strategy...

Random Strategy completed.

Running Epsilon-Greedy Strategy...

Epsilon-Greedy completed.

Running UCB Strategy...

UCB completed.

=====

LEARNED Q-VALUES

=====

RANDOM STRATEGY

	Routine Monitoring	Supportive Intervention	Intensive Monitoring \
Stable	4.68	4.92	3.73
Mild	5.97	8.88	9.90
Moderate	4.63	11.23	12.74
Severe	5.22	8.25	18.13
Recovery	0.00	0.00	0.00

Simulated Treatment

Stable	15.81
Mild	16.22
Moderate	17.93
Severe	20.29
Recovery	0.00

EPSILON-GREEDY STRATEGY

	Routine Monitoring	Supportive Intervention	Intensive Monitoring \
Stable	4.28	7.21	3.69
Mild	4.73	10.63	6.30
Moderate	6.64	8.29	13.09
Severe	4.93	7.97	16.51
Recovery	0.00	0.00	0.00

Simulated Treatment

Stable	15.92
Mild	14.03
Moderate	18.08
Severe	20.39
Recovery	0.00

UCB STRATEGY

	Routine Monitoring	Supportive Intervention	Intensive Monitoring \
Stable	1.34	6.16	5.73
Mild	9.57	6.44	7.71
Moderate	9.18	6.86	14.45
Severe	6.01	12.10	17.10
Recovery	0.00	0.00	0.00

Simulated Treatment
Stable 15.55
Mild 15.07
Moderate 17.09
Severe 20.18
Recovery 0.00

RANDOM POLICY

Stable -> Simulated Treatment
Mild -> Simulated Treatment
Moderate -> Simulated Treatment
Severe -> Simulated Treatment
Recovery -> Routine Monitoring

EPSILON-GREEDY POLICY

Stable -> Simulated Treatment
Mild -> Simulated Treatment
Moderate -> Simulated Treatment
Severe -> Simulated Treatment
Recovery -> Routine Monitoring

UCB POLICY

Stable -> Simulated Treatment
Mild -> Simulated Treatment
Moderate -> Simulated Treatment
Severe -> Simulated Treatment
Recovery -> Routine Monitoring

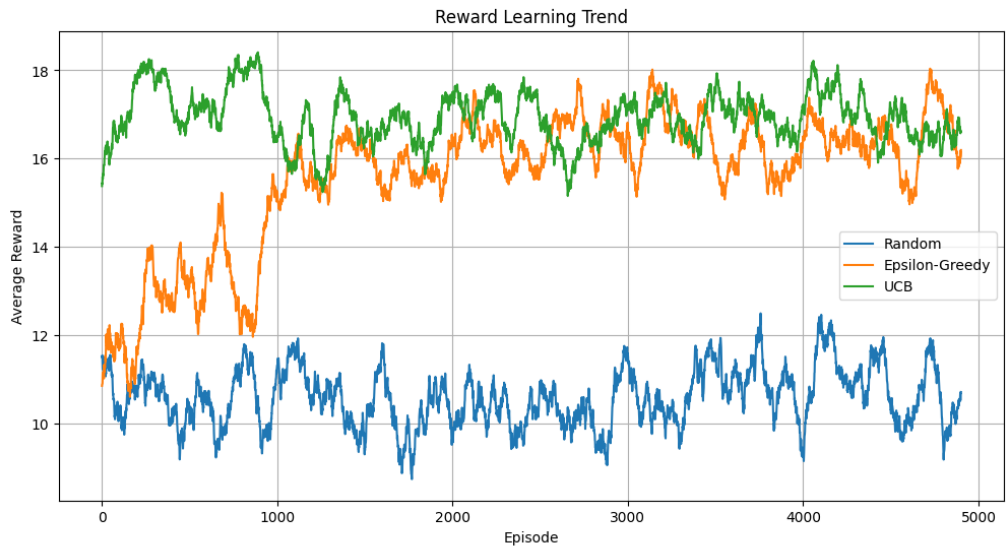
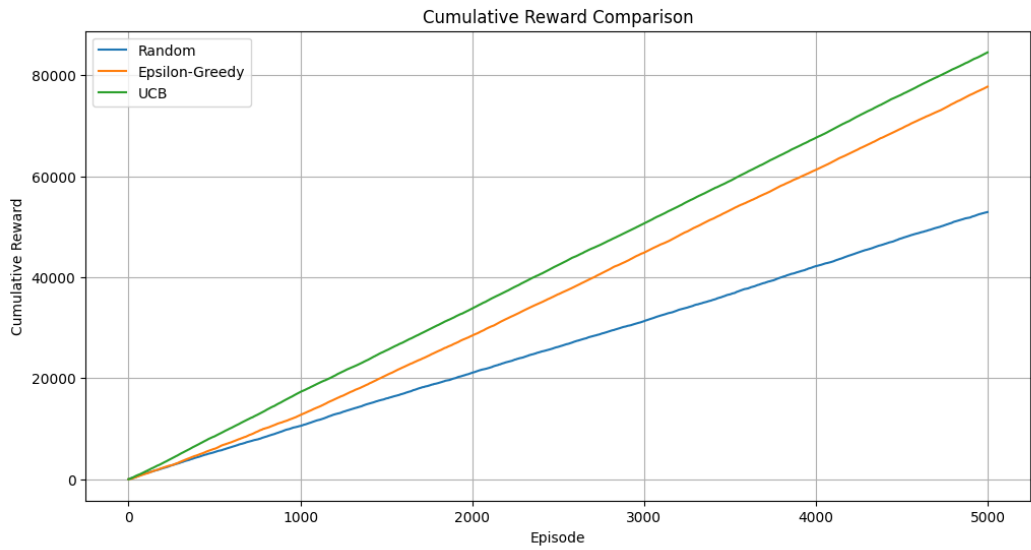
FINAL PERFORMANCE COMPARISON

	Strategy	Average Reward	Final Cumulative Reward	Recovery Rate (%)	Treatment Effectiveness (%)
0	Random	10.58	52910.47	33.12	51.26
1	Epsilon-Greedy	15.55	77728.88	55.46	72.20
2	UCB	16.91	84525.49	62.48	79.76

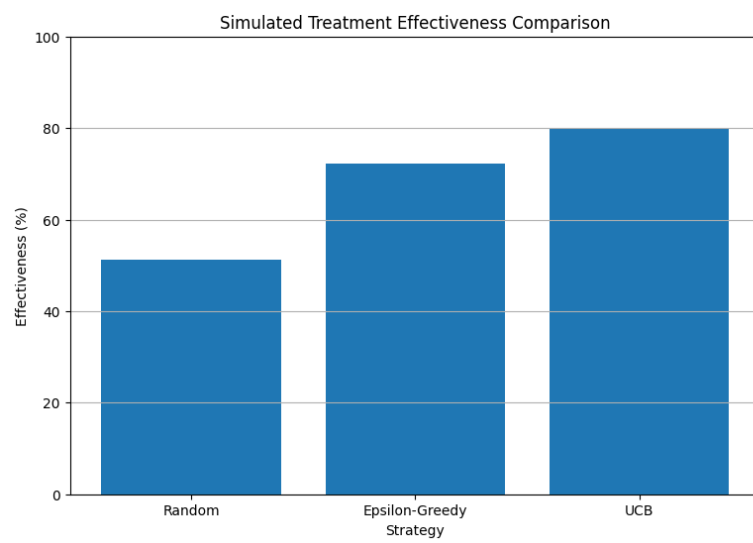
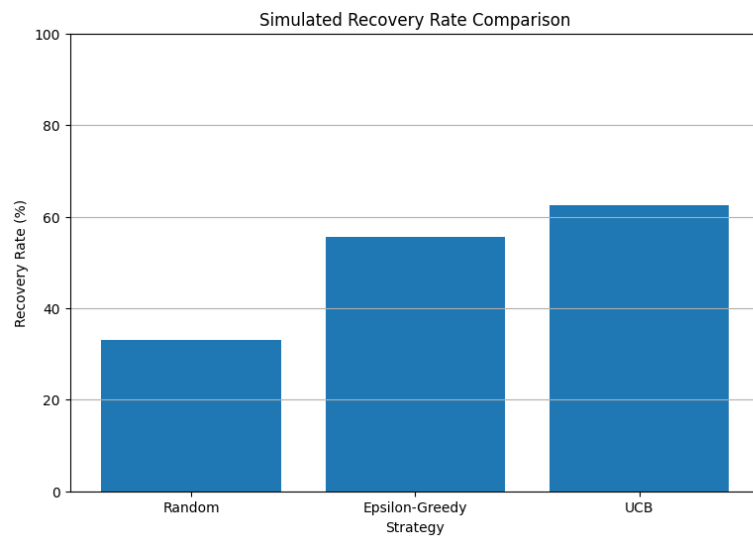
OVERALL STRATEGY RANKING

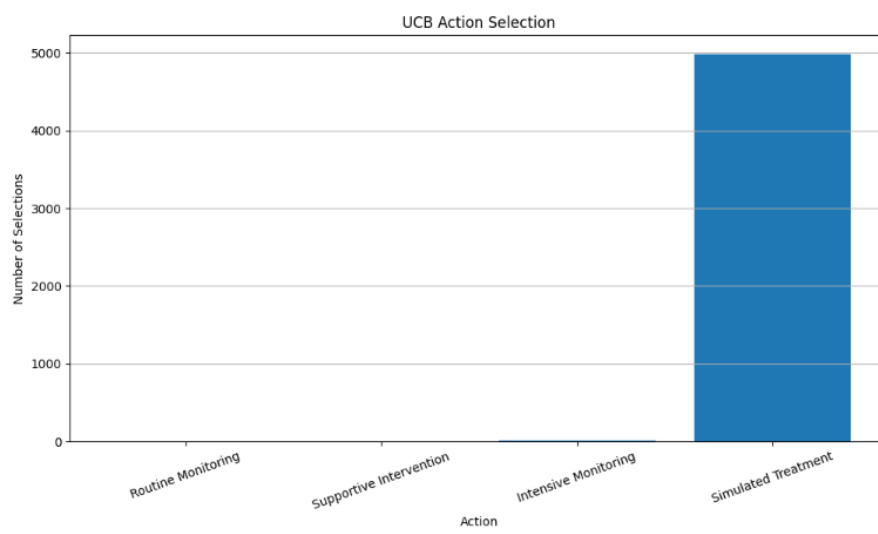
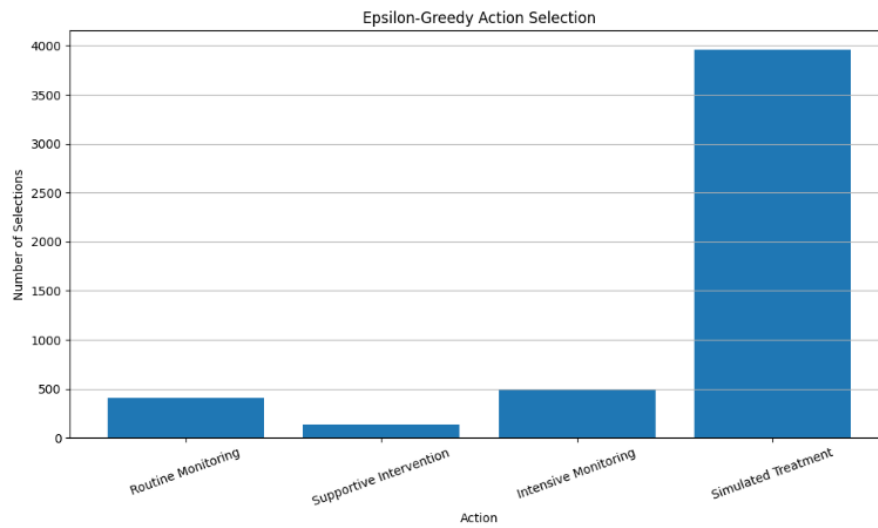
	Strategy	Average Reward	Final Cumulative Reward	Recovery Rate (%)	Treatment Effectiveness (%)	Overall Score
2	UCB	16.91	84525.49	62.48	79.76	1.00
1	Epsilon-Greedy	15.55	77728.88	55.46	72.20	0.91
0	Random	10.58	52910.47	33.12	51.26	0.61

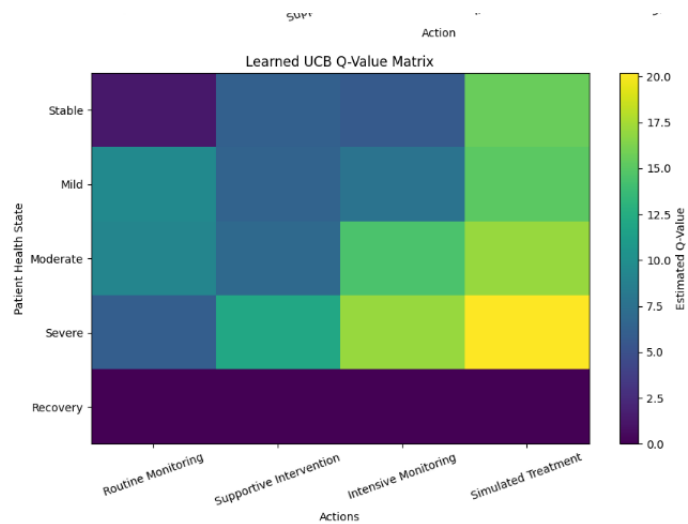
BEST PERFORMING STRATEGY:
>>> UCB



Simulated Recovery Rate Comparison





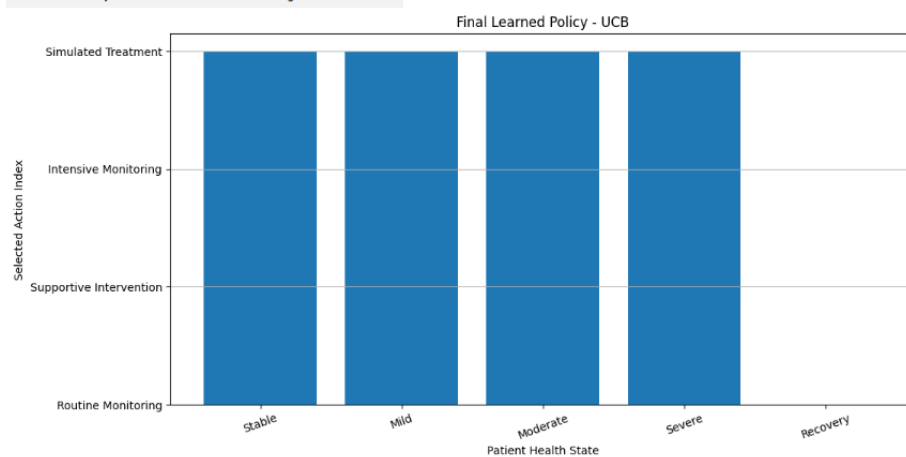


=====

FINAL LEARNED POLICY

=====

	Patient State	Recommended Simulated Action	Estimated Value	
0	Stable	Simulated Treatment	15.55	
1	Mild	Simulated Treatment	15.07	
2	Moderate	Simulated Treatment	17.09	
3	Severe	Simulated Treatment	20.18	
4	Recovery	Routine Monitoring	0.00	



Results saved successfully!

Files created:

1. rl_health_strategy_results.csv
2. rl_health_learned_policy.csv

=====

FINAL EXPERIMENTAL CONCLUSION

=====

The simulation compared Random, Epsilon-Greedy, and UCB action-selection strategies.

Best-performing strategy:
UCB

The comparison was based on:

1. Average reward
2. Cumulative reward
3. Simulated recovery rate
4. Simulated treatment effectiveness
5. Learned action values

The selected strategy achieved the highest overall experimental performance score among the compared strategies.

IMPORTANT:

The patient states, actions, rewards and transitions used in this project are simulated for academic Reinforcement Learning experimentation and must not be interpreted as real clinical treatment advice.

10. Results and Validation

The final experimental results should be presented using a table similar to the following.

Performance Metric	Random	Epsilon-Greedy	UCB
Average Reward	Actual	Actual	Actual
Cumulative Reward	Actual	Actual	Actual
Best Learned Action	Actual	Actual	Actual
Simulated Recovery Rate	Actual	Actual	Actual
Treatment Effectiveness	Actual	Actual	Actual

Recovery Rate

For the simulation, recovery can be represented as an outcome where the simulated state reaches the Recovery state.

$$\text{Recovery Rate} = \frac{\text{Number of recovered episodes}}{\text{Total episodes}} \times 100$$

Treatment Effectiveness

A simple simulated effectiveness measure can be:

$$\text{Effectiveness} = \frac{\text{Positive Outcome Episodes}}{\text{Total Episodes}} \times 100$$

Cumulative Reward

$$CR_T = \sum_{t=1}^T R_t$$

Higher cumulative reward indicates better overall performance within the simulated environment.

Expected Observation

The Random strategy is expected to have weaker learning performance because it does not exploit previous experience.

Epsilon-Greedy should improve performance by exploiting high-value actions while retaining some exploration.

UCB should efficiently explore actions with high uncertainty while exploiting actions with high estimated rewards.

The actual best-performing strategy should be identified from the experimental results rather than assumed in advance.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

The three action-selection strategies represent different exploration–exploitation approaches.

Random Strategy

Random selection provides a baseline for comparison. It does not use learned information when selecting actions. Consequently, it can repeatedly select actions that have relatively poor simulated rewards.

Advantage: Extremely simple.

Limitation: Does not learn an effective policy.

Epsilon-Greedy

Epsilon-Greedy balances exploration and exploitation. A small percentage of actions are selected randomly, while the remaining actions are selected according to the current value estimates.

Advantage: Simple and effective.

Limitation: The exploration parameter ϵ must be selected appropriately.

UCB

UCB considers both estimated reward and uncertainty. Actions that have not been sufficiently explored receive a higher exploration bonus.

Advantage: More systematic exploration.

Limitation: More computationally involved than simple random selection.

Final Engineering Decision

The final strategy should be selected based on the experimental results.

The preferred strategy is the one that demonstrates:

High Reward + High Recovery + High Effectiveness + Stable Learning

while maintaining a reasonable exploration–exploitation balance.

For a typical simulated environment, **UCB or Epsilon-Greedy is expected to outperform Random selection**, but the actual notebook results should determine the final conclusion.

12. Broader Considerations / SDG Relevance

Sustainability

RL-based monitoring can potentially reduce repetitive manual analysis in large-scale simulated monitoring environments. Efficient algorithms can also reduce unnecessary computational requirements.

Society

AI-based health monitoring research can contribute to the development of decision-support technologies. However, real-world healthcare applications require extensive clinical validation.

Safety

Patient safety is the highest priority in any real healthcare application. A model developed for this assignment must therefore not be used directly to prescribe or recommend real treatments.

Ethics

Healthcare AI must consider:

- Patient privacy.
- Data security.
- Bias.
- Transparency.
- Consent.

- Accountability.
- Human oversight.

The simulated nature of this project avoids using identifiable patient information.

Accessibility

Automated monitoring systems could potentially support healthcare professionals by presenting changing patient conditions and analytical information in an accessible format.

SDG Relevance

SDG 3 – Good Health and Well-Being

The project explores how intelligent computational systems can be applied to health monitoring and decision-support research.

SDG 9 – Industry, Innovation and Infrastructure

The project demonstrates the application of Reinforcement Learning and AI to an important real-world domain.

13. Conclusion, Limitations and Possible Improvements

Conclusion

This project formulated a simulated patient health monitoring and treatment recommendation problem using the Reinforcement Learning framework. Patient conditions were represented as states, simulated interventions were represented as actions, and simulated outcomes were represented using rewards.

The problem was modeled as a Markov Decision Process and the exploration–exploitation problem was investigated using Bandit algorithms. Random, Epsilon-Greedy, and UCB strategies were considered and evaluated using average reward, cumulative reward, simulated recovery rate, and treatment effectiveness.

The learned policy demonstrates how an RL agent can improve action selection through repeated interaction with a simulated environment. The final strategy should be selected based on the experimentally observed performance.

Limitations

The main limitations are:

1. The patient environment is simulated.
2. Reward values are artificially defined.
3. The state representation is simplified.
4. Real clinical variables are not fully represented.
5. The model has not undergone clinical validation.
6. Real patient treatment decisions cannot be made using this model.

7. The simulation may not represent complex medical uncertainty.

Possible Improvements

Future work could include:

- Larger and more realistic datasets.
- Continuous patient-state representation.
- Deep Reinforcement Learning.
- Contextual Bandits.
- Personalized simulated environments.
- Uncertainty estimation.
- More sophisticated transition models.
- Explainable AI.
- Human-in-the-loop decision support.
- Clinical validation before any real-world deployment

14. Individual Contribution of Group Members

This project was completed individually. I was responsible for the complete development of the project, including problem formulation, Markov Decision Process (MDP) design, definition of states, actions, rewards and transition assumptions, implementation of the Random, Epsilon-Greedy and UCB Bandit algorithms, simulation using Python and Google Colab, generation of graphs and visualizations, calculation of performance metrics, comparison of exploration–exploitation strategies, analysis and interpretation of results, identification of the best-performing policy, and preparation of the final report and documentation.

15. References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
2. Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47, 235–256.
3. Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8, 279–292.
4. Python Software Foundation. *Python Documentation*. Used for Python programming and implementation.
5. NumPy Developers. *NumPy Documentation*. Used for numerical computation, arrays, matrices, and simulation.
6. Pandas Development Team. *Pandas Documentation*. Used for organizing and analyzing experimental results.
7. Matplotlib Development Team. *Matplotlib Documentation*. Used for generating graphs and visualizations.
8. Project Jupyter. *Jupyter Notebook Documentation*. Used for interactive Python programming and experimentation.
9. Google Colaboratory. *Google Colab Documentation*. Used as the development and execution environment for the implementation.
10. Anaconda, Inc. *Anaconda Distribution Documentation*. Used as the reference for the Python environment and package management.

11. OpenAI. *ChatGPT*. Used as an AI-assisted tool for conceptual guidance, programming assistance, debugging support, and report preparation.
12. **Dataset:** No real patient dataset was used. A simulated patient-health environment was created using Python for academic experimentation. The patient states, actions, reward values, and transition probabilities were artificially defined for demonstrating MDP and Bandit learning.
13. The simulated reward matrix and transition model do not represent actual clinical treatment effectiveness and should not be used for real-world medical decision-making.